



POLITECNICO DI MILANO
DEPARTMENT OF ELECTRONICS, INFORMATION AND BIOENGINEERING

ICL-BLDC

modifications

Giuseppe Fiengo

✉ giuseppe.fiengo@polimi.it

🔗 [ICL-BLDC](#)

📅 January 27, 2026

Contents

1	Learning Setup and Data Processing	1
1.1	Loading DataSets	1
1.2	Time-Weighted Mean Squared Error Loss	3
1.3	Resource Monitoring and Compute Profiling	4
2	Training Process	5
2.1	Autoregressive Regression Step	5
2.2	Smoothness Regularisation via Finite Differences	6
3	Transformer Module Design	9
3.1	Activation Function in the Feed-Forward Network	9
4	Proposed Improvements and Extensions	11
4.1	Mitigating Training-Inference Mismatch	11
4.2	Model-Level Enhancements	11
4.3	Evaluation Strategy and Test-Time Monitoring	12
4.4	Scheduled Sampling for Teacher-Free Autoregressive Training	13
4.4.1	Autoregressive training setup	13
4.4.2	Scheduled sampling formulation	13
4.4.3	Teacher probability schedules	14
4.4.4	Implementation in the training loop	14
4.4.5	Deterministic alternative	14
4.4.6	Status	14
5	Hyperparameter Sensitivity and Trade-off Analysis	17
5.1	Purpose and Scope	17
5.2	Experimental Setup	17
5.3	Global Relationships Between Hyperparameters, Loss, and Time	17
5.4	One-at-a-Time Sensitivity Analysis	18
5.4.1	Quantitative Summary	18
5.4.2	Slice-Based Visualization	18
5.5	Trade-off Analysis and Pareto Optimality	19
5.5.1	Pareto Front	19
5.5.2	Weighted Trade-off Selection	20
5.6	Discussion	21
5.7	Ablation on Regression Mode, Activation Function, and Smoothness	22
	Bibliography	23

1 Learning Setup and Data Processing

1.1 Loading DataSets

Algorithm 1 Dataset Configuration and Loading

Require: Configuration dictionary $\mathcal{C}$

Ensure: Cached data pools and normalized folder definitions

```
1: Extract split ratios ( $r_{\text{train}}, r_{\text{val}}, r_{\text{test}}$ )
2: if  $r_{\text{train}} + r_{\text{val}} + r_{\text{test}} \neq 1$  then
3:   raise error
4: end if

5: Read folder lists ( $F_{\text{train}}, F_{\text{val}}, F_{\text{test}}$ ) from  $\mathcal{C}$ 
6: if all folder lists are undefined then ▷ Backward compatibility
7:   Set  $F_{\text{train}} = F_{\text{val}} = F_{\text{test}} = F_{\text{base}}$ 
8: else
9:   Fill missing folder lists by reusing available definitions
10: end if

11: Convert folder lists to sets:
      
$$S_{\text{train}}, S_{\text{val}}, S_{\text{test}}$$


12: Identify all unique folders:
      
$$S_{\text{all}} = S_{\text{train}} \cup S_{\text{val}} \cup S_{\text{test}}$$


13: for each folder  $f \in S_{\text{all}}$  do
14:   Load all data files from  $f$ 
15:   Cache loaded files in memory
16: end for

17: Initialize deterministic random generator with fixed seed
18: return cached data pools and normalized folder sets
```

Algorithm 2 Dataset Splitting Logic

Require: Cached data pools and folder sets ($S_{\text{train}}, S_{\text{val}}, S_{\text{test}}$)**Ensure:** Datasets ($\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$)

- 1: **if** $S_{\text{train}} = S_{\text{val}} = S_{\text{test}}$ **then** ▷ Original behavior
 - 2: Collect all files into a single pool $\mathcal{P}$
 - 3: Shuffle $\mathcal{P}$
 - 4: Split $\mathcal{P}$ into

$\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$
 - 5: **using** ($r_{\text{train}}, r_{\text{val}}, r_{\text{test}}$)
 - 6: **else if** exactly two of the folder sets are equal **then** ▷ Priority-based partial split
 - 7: Assign the distinct folder set entirely to its dataset
 - 8: Merge data from the two equal folder sets into pool $\mathcal{P}$
 - 9: Shuffle $\mathcal{P}$
 - 10: Split $\mathcal{P}$ according to priority:
 - Train keeps r_{train} if involved
 - Validation keeps r_{val} if only val/test are involved
 - 11: **else** ▷ Fully disjoint folders
 - 12: Collect datasets independently:

$\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$
 - 13: Shuffle each dataset independently
 - 14: **end if**
 - 15: Construct sequence datasets with fixed sequence length
 - 16: **return** ($\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$)
-

1.2 Time-Weighted Mean Squared Error Loss

Let $y_{\text{pred}}, y_{\text{true}} \in \mathbb{R}^{B \times T \times D}$ denote, respectively, the predicted and ground-truth output sequences, where B is the batch size, T the sequence length, and D the output dimension.

Standard Mean Squared Error. The standard mean squared error (MSE) loss over a sequence is defined as

$$\mathcal{L}_{\text{MSE}} = \frac{1}{BTD} \sum_{b=1}^B \sum_{t=1}^T \sum_{d=1}^D \left(y_{\text{pred}}^{(b,t,d)} - y_{\text{true}}^{(b,t,d)} \right)^2. \quad (1.1)$$

This formulation assigns equal importance to all time steps in the sequence. While this choice is simple and stable, it implicitly assumes that prediction errors are equally relevant across the entire temporal horizon.

Time-Weighted MSE Formulation. To explicitly emphasize prediction accuracy at later time steps, a time-weighted variant of the MSE is adopted. First, a per-time-step MSE is computed as

$$\text{MSE}_t^{(b)} = \frac{1}{D} \sum_{d=1}^D \left(y_{\text{pred}}^{(b,t,d)} - y_{\text{true}}^{(b,t,d)} \right)^2. \quad (1.2)$$

A set of non-negative temporal weights $\{w_t\}_{t=1}^T$ is then defined according to one of the following schemes:

$$w_t = \frac{t}{T}, \quad (\text{linear weighting}) \quad (1.3)$$

$$w_t = \exp\left(\alpha \frac{t-1}{T-1}\right), \quad (\text{exponential weighting}) \quad (1.4)$$

where $\alpha > 0$ controls the strength of the exponential weighting.

To ensure scale invariance with respect to the sequence length, the weights are normalized such that

$$\tilde{w}_t = \frac{w_t}{\sum_{k=1}^T w_k}. \quad (1.5)$$

The resulting time-weighted MSE loss is defined as

$$\mathcal{L}_{\text{TW-MSE}} = \frac{1}{B} \sum_{b=1}^B \sum_{t=1}^T \tilde{w}_t \text{MSE}_t^{(b)}. \quad (1.6)$$

Discussion. Compared to the standard MSE, the proposed loss introduces an explicit inductive bias that prioritizes accuracy at later time steps in the sequence. This choice is motivated by scenarios in which early predictions mainly serve as context, while the quality of long-horizon predictions is more critical for evaluation and downstream use.

The main advantage of this formulation is improved alignment with long-horizon performance metrics. However, it may also reduce gradient contributions from early time steps and increase sensitivity to noise at later steps. Consequently, the weighting strategy and the parameter α must be selected with care, and the standard MSE remains a relevant baseline when uniform temporal accuracy or training stability is the primary concern.

1.3 Resource Monitoring and Compute Profiling

To characterize the computational cost of training and to support reproducibility across machines and runs, a lightweight `ResourceMonitor` utility was added. The monitor periodically samples system-, process-, and (when available) GPU-level metrics during training and stores them as a time series. This enables post-hoc analysis of whether a given configuration is compute-bound (GPU utilization saturated) or input/CPU/I/O-bound (low GPU utilization with high CPU, disk, or RAM activity).

Logged quantities. At each sample time, the monitor records:

- **System CPU and RAM:** CPU utilization, total/available memory, and memory percentage.
- **Process statistics:** resident memory (RSS), virtual memory (VMS), number of threads, and process CPU utilization.
- **I/O deltas:** network and disk read/write bytes since monitoring started.
- **GPU statistics (if NVML is available):** GPU utilization, memory usage, temperature, power draw and power limit, SM/memory clock, and (when supported) ECC error counters.

GPU metrics are obtained via NVIDIA’s NVML interface when available; otherwise, the monitor logs `NaN` values while keeping the same schema, ensuring consistent CSV/DataFrame columns across different hardware setups.

Design choices and parameter rationale. The monitor is designed to be minimally intrusive: sampling is performed in a background daemon thread and uses a fixed sampling interval

$$\Delta t = 5 \text{ s}$$

(`sample_interval_s = 5.0`). This interval was chosen as a compromise between temporal resolution and overhead. Training dynamics (e.g., steady-state GPU utilization, memory growth, thermal or power throttling) typically evolve over timescales of seconds to minutes rather than milliseconds; therefore, a 5-second interval provides sufficient fidelity for diagnosing bottlenecks without significantly perturbing training throughput or generating excessively large logs.

The parameter `device_index = 0` fixes the monitored GPU to the primary device, matching the default single-GPU training configuration. Finally, the switch `enabled` allows disabling monitoring entirely to guarantee that experiments can be run with zero monitoring overhead when desired.

Practical motivation. This profiling step was introduced because the proposed training procedure includes computationally heavier components (e.g., autoregressive unrolling over a window and additional derivative-based regularization). Resource monitoring makes it possible to empirically verify how these design choices impact compute utilization, memory footprint, and I/O behavior, and to distinguish genuine model-side slowdowns from data-loading or system-level bottlenecks.

2 Training Process

2.1 Autoregressive Regression Step

Training is performed over mini-batches of input-output windows of fixed length H . Each batch contains

$$u \in \mathbb{R}^{B \times H \times F}, \quad y \in \mathbb{R}^{B \times H \times 1},$$

where B is the batch size and F is the number of input features. The model is trained in an autoregressive (teacher-free) manner: the predicted speed $\hat{\omega}$ is recursively fed back into the input at the subsequent time step, in order to match the conditions encountered at inference time.

New implementation. Let $f_\theta(\cdot)$ denote the Transformer model with parameters θ . Before the recursion, the input channel corresponding to the speed signal (channel index 4 in the implementation) is set to zero, ensuring that the model cannot trivially copy the ground-truth value. The regression procedure then generates predictions sequentially as:

$$\hat{\omega}_0 := 0, \tag{2.1}$$

$$\hat{\omega}_t := f_\theta(\tilde{u}_{1:t})_{\text{last}}, \quad t = 1, \dots, H, \tag{2.2}$$

where $\tilde{u}_{1:t}$ denotes the strictly-causal prefix input sequence (from time 1 to t) in which the speed channel at time t has been overwritten with the previous estimate $\hat{\omega}_{t-1}$, and $(\cdot)_{\text{last}}$ indicates that the model output is taken at the last time position. The resulting prediction window is assembled as

$$\hat{y} = [\hat{\omega}_1, \hat{\omega}_2, \dots, \hat{\omega}_H] \in \mathbb{R}^{B \times H \times 1}. \tag{2.3}$$

The primary data-fit term is computed as

$$\mathcal{L}_{\text{fit}} = \mathcal{C}(y, \hat{y}), \tag{2.4}$$

where $\mathcal{C}(\cdot, \cdot)$ is the chosen regression criterion (e.g., MSE or time-weighted MSE).

A key architectural change relative to the earlier implementation is that the regression logic is factored out into a dedicated function `regression( $\cdot$ )`, controlled through the parameter `regression_mode`. In the default mode (`time_last`) the behavior corresponds to the explicit recursion in (2.2), producing one prediction per time step by repeatedly running the model on progressively longer prefixes.

Old implementation. In the original training loop, the regression step was implemented inline inside the `train` function. Functionally, the autoregressive recursion was the same: the speed channel was zeroed and then overwritten at each step with the previous predicted value, and the prediction sequence $\hat{y}$ was constructed by concatenating last-token predictions across time. The main difference is that the earlier code coupled this regression recursion with the smoothness penalty by concatenating additional signals into the regression loss itself (see Section 2.2), rather than treating smoothness as a separate additive term.

Practical consequence. The new implementation cleanly separates: (i) autoregressive sequence generation (regression), and (ii) objective design (data-fit and smoothness). This makes it possible to compare regression modes and regularization terms without changing the core training loop structure.

2.2 Smoothness Regularisation via Finite Differences

In addition to the data-fit loss, a smoothness term is optionally introduced to discourage unrealistic step-to-step variations in the predicted speed trajectory. This is motivated by the fact that physically plausible motor speed dynamics are typically smooth at the sampling rates used in the dataset; therefore, penalising abrupt variations can improve trajectory realism and reduce high-frequency artefacts.

Let the discrete first difference operator along time be defined as

$$\Delta y_t := y_t - y_{t-1}, \quad t = 2, \dots, H, \quad (2.5)$$

applied independently for each element in the batch. In tensor form, $\Delta y \in \mathbb{R}^{B \times (H-1) \times 1}$.

New smoothness formulation: dynamics-matching penalty. The new training procedure introduces a derivative matching term comparing the first differences of predicted and true trajectories:

$$\Delta y := \text{diff}(y, \text{time}), \quad (2.6)$$

$$\Delta \hat{y} := \text{diff}(\hat{y}, \text{time}). \quad (2.7)$$

A weighted target derivative is then defined as

$$\Delta y_{\text{target}} := w_{\text{dyn}} \odot \Delta y, \quad (2.8)$$

where $\odot$ denotes elementwise multiplication and $w_{\text{dyn}} \in [0, 1]^{B \times (H-1) \times 1}$ is a data-dependent weight computed from the magnitude of the true derivative:

$$w_{\text{dyn}} := \text{clip}\left(\frac{|\Delta y|}{\tau}, 0, 1\right), \quad (2.9)$$

with threshold $\tau > 0$ (fixed in the implementation) and $\text{clip}(\cdot)$ clamping the value into $[0, 1]$.

The resulting smoothness loss is

$$\mathcal{L}_{\text{dyn}} = \|\Delta \hat{y} - \Delta y_{\text{target}}\|_2^2 = \frac{1}{B(H-1)} \sum_{b=1}^B \sum_{t=2}^H \left(\Delta \hat{y}_t^{(b)} - \Delta y_{\text{target},t}^{(b)} \right)^2. \quad (2.10)$$

The total loss is then

$$\mathcal{L} = \mathcal{L}_{\text{fit}} + R_{\text{smooth}}^2 \mathcal{L}_{\text{dyn}}, \quad (2.11)$$

where $R_{\text{smooth}} \geq 0$ controls the strength of the regularization.

Interpretation. Unlike a generic smoothness constraint pushing derivatives toward zero, this term promotes *dynamics matching*: the predicted derivative is encouraged to follow the ground-truth derivative when the latter is significant, while being weakly constrained when the true derivative is near zero due to the weighting w_{dyn} .

Old smoothness formulation: derivative-to-zero augmentation. The previous training code implemented smoothness by augmenting the regression target and prediction arrays. The predicted derivative was computed as $\Delta\hat{y}$ and multiplied by a scalar R , after which the objective enforced this scaled derivative to be close to zero by concatenating it to the original regression target:

$$\hat{y}_{\text{aug}} := [\hat{y}, R \Delta\hat{y}], \quad (2.12)$$

$$y_{\text{aug}} := [y, 0], \quad (2.13)$$

and then applying the base criterion:

$$\mathcal{L}_{\text{old}} = \mathcal{C}(y_{\text{aug}}, \hat{y}_{\text{aug}}). \quad (2.14)$$

Key difference in the derivative definition. Both approaches use first differences of the predicted sequence; however, they differ in the *target* for those differences:

- **Old approach:** forces $R \Delta\hat{y}$ toward zero, encouraging constant outputs irrespective of the true dynamics.
- **New approach:** encourages $\Delta\hat{y}$ to match (a weighted version of) Δy , enabling changes when the true trajectory changes.

Advantages and disadvantages.

New dynamics-matching penalty.

- **Advantage:** better physical plausibility, since the model is not penalized for changing output when the underlying system truly changes.
- **Advantage:** reduces oversmoothing artifacts (flat predictions) that can arise when enforcing $\Delta\hat{y} \approx 0$.
- **Disadvantage:** introduces additional hyperparameters (threshold τ and regularization strength R_{smooth}) and can amplify the impact of noisy derivatives at high sampling rates.

Old derivative-to-zero formulation.

- **Advantage:** simple regularizer that can reduce high-frequency jitter and stabilize training early.
- **Disadvantage:** structurally biases the solution toward constant trajectories, which can hurt performance when the motor speed profile contains genuine accelerations/decelerations.

Implementation note. In the new formulation, the smoothness penalty is explicitly additive (Eq. (2.11)), decoupled from the regression criterion $\mathcal{C}$. This simplifies ablation studies: $\mathcal{L}_{\text{fit}}$ can be changed (e.g., MSE vs time-weighted MSE) without modifying the definition of the smoothness term.

3 Transformer Module Design

3.1 Activation Function in the Feed-Forward Network

Each Transformer block includes a position-wise feed-forward network (FFN) that performs non-linear feature mixing after the self-attention operation. In the implemented architecture, this module follows the standard two-layer structure:

$$\mathbf{x} \mapsto \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}), \quad (3.1)$$

where $\sigma(\cdot)$ is an element-wise activation function.

Initial choice: GELU. The initial implementation adopted the Gaussian Error Linear Unit (GELU) activation, defined as

$$\text{GELU}(x) = x \Phi(x), \quad (3.2)$$

where $\Phi(\cdot)$ denotes the cumulative distribution function of a standard normal variable. GELU is the default activation used in many Transformer architectures, including the original GPT models, and is known to provide smooth, non-linear gating of activations.

Adopted alternative: SiLU. In the present work, the activation function was generalized and the Sigmoid Linear Unit (SiLU), also known as the Swish activation, was preferred:

$$\text{SiLU}(x) = x \cdot \sigma(x), \quad (3.3)$$

where $\sigma(\cdot)$ denotes the logistic sigmoid.

The feed-forward network is therefore implemented as:

$$\text{FFN}(\mathbf{x}) = \text{Dropout}(\mathbf{W}_2 \text{SiLU}(\mathbf{W}_1 \mathbf{x})). \quad (3.4)$$

Motivation and discussion. The choice of SiLU over GELU is motivated by the following considerations:

- **Smooth gradients:** SiLU is continuously differentiable and exhibits non-zero gradients for negative inputs, which can improve gradient flow in deep networks.
- **Stable behavior for regression:** In contrast to ReLU-based activations, SiLU avoids hard saturation and dead units, which is beneficial in regression-oriented tasks such as continuous motor speed prediction.
- **Empirical robustness:** SiLU has shown favorable empirical performance in several modern architectures, particularly in settings where smooth input-output mappings are desired.

While GELU remains a strong and widely adopted default, the use of SiLU introduces a slightly less aggressive gating mechanism, which is better aligned with the continuous and physically smooth nature of the modeled dynamics. For this reason, SiLU was selected as the preferred activation function in the Transformer feed-forward blocks, while retaining the option to switch activations through configuration for ablation and comparison purposes.

4 Proposed Improvements and Extensions

This chapter discusses additional improvements to the learning pipeline that were identified after defining the new regression logic, smoothness regularization, and training process. These improvements primarily address training–inference mismatch, model expressiveness, and evaluation robustness, rather than introducing new architectural components.

4.1 Mitigating Training–Inference Mismatch

The autoregressive training procedure feeds the model with its own previous predictions rather than ground-truth outputs. While this teacher-free setup improves robustness at inference time, it also makes optimization more challenging, especially in early training stages when prediction errors are large and compound rapidly.

A common mitigation strategy is *scheduled sampling*, where the model is intermittently conditioned on ground-truth outputs during training. Formally, at each time step t , the injected value is defined as

$$\tilde{y}_{t-1} = \begin{cases} y_{t-1}, & \text{with probability } p(t), \\ \hat{y}_{t-1}, & \text{with probability } 1 - p(t), \end{cases} \quad (4.1)$$

where $p(t)$ is a schedule that decreases over training epochs.

Motivation. Scheduled sampling reduces gradient variance and stabilizes early optimization, while still converging toward a fully autoregressive regime. In practice, this can lead to improved long-horizon predictions without sacrificing training stability.

Status. In the present implementation, fully teacher-free autoregression is used. Scheduled sampling is identified as a natural extension to further improve convergence behaviour. More in Section 4.4.

4.2 Model-Level Enhancements

Relative Temporal Encoding. The current Transformer implementation uses absolute positional embeddings. While effective, absolute encodings implicitly bind dynamics to fixed time indices within the window. An alternative is to adopt relative positional encodings, which emphasize temporal differences rather than absolute position.

Expected benefit. Relative encodings can improve generalization across windows of different lengths or across shifted temporal contexts, which is particularly relevant for dynamical systems where relative timing matters more than absolute index.

Predicting Increments Instead of Absolute Values. An additional model-level improvement consists in predicting output increments rather than absolute values:

$$\Delta \hat{y}_t = f_\theta(\cdot), \quad \hat{y}_t = \hat{y}_{t-1} + \Delta \hat{y}_t. \quad (4.2)$$

Motivation. Incremental prediction often improves numerical stability and reduces drift in autoregressive rollouts, as the model focuses on local dynamics rather than reconstructing absolute magnitudes at each step. This formulation also aligns naturally with derivative-based regularization.

4.3 Evaluation Strategy and Test-Time Monitoring

A separate test dataset split is constructed during data preprocessing. Evaluation on the test split is performed outside the training loop, using a dedicated evaluation function that mirrors the inference-time rollout.

Role of the test step. Importantly, incorporating a test step during training does *not* influence the learned model parameters. No gradients are computed, and no optimizer updates are performed based on test-set performance.

What changes. Adding periodic test evaluation during training enables:

- tracking generalization error over training epochs;
- detecting overfitting or performance degradation early;
- comparing regression modes and regularization strategies under identical evaluation conditions;
- analyzing long-horizon rollout behavior on unseen trajectories.

What does not change. The test step does *not*:

- modify network weights or biases;
- alter the training objective;
- improve predictions by itself.

Why this still matters. Although test evaluation does not directly affect optimization, it plays a critical role in experimental correctness. Without systematic test-time monitoring, improvements in training loss may not translate to improved predictive performance under inference conditions, particularly for autoregressive models where small errors can accumulate over time.

Summary. The inclusion of structured evaluation and monitoring improves the reliability and interpretability of experimental results, even though it does not directly intervene in the parameter update process.

4.4 Scheduled Sampling for Teacher-Free Autoregressive Training

4.4.1 Autoregressive training setup. Consider input–output sequences of fixed horizon H , with

$$u_{1:H} \in \mathbb{R}^{H \times d_u}, \quad y_{1:H} \in \mathbb{R}^H.$$

The model is trained autoregressively over each window, producing predictions $\hat{y}_t$ recursively for $t = 1, \dots, H$. At each step, the model is conditioned on all past inputs and an injected value representing the previous output.

In the current implementation, training is performed in a fully teacher-free manner. Specifically, the injected value at time t is given by

$$\tilde{y}_{t-1} = \hat{y}_{t-1}, \quad \hat{y}_0 := 0,$$

so that the model always feeds back its own predictions. Formally, the forward recurrence can be written as

$$\hat{y}_t = f_\theta(u_{1:t}, \tilde{y}_{0:t-1}), \quad t = 1, \dots, H,$$

where θ denotes the trainable parameters.

The training objective over a window is

$$\mathcal{L}(\theta) = \frac{1}{H} \sum_{t=1}^H \ell(y_t, \hat{y}_t) + \lambda \mathcal{L}_{\text{smooth}}(y_{1:H}, \hat{y}_{1:H}), \quad (4.3)$$

where $\ell(\cdot, \cdot)$ is typically the mean squared error and $\mathcal{L}_{\text{smooth}}$ is an additional regularization term penalizing rapid temporal variations in the predicted trajectory.

This training strategy reduces the discrepancy between training and inference distributions, since inference is also performed in a self-feeding autoregressive regime. However, it significantly increases optimization difficulty in early training stages, as prediction errors can accumulate and propagate through the input recursion.

4.4.2 Scheduled sampling formulation. To mitigate early training instability, *scheduled sampling* introduces stochastic teacher forcing during training. At each time step t and training epoch e , the injected value is defined as

$$\tilde{y}_{t-1} = b_{t,e} y_{t-1} + (1 - b_{t,e}) \hat{y}_{t-1}, \quad b_{t,e} \sim \text{Bernoulli}(p_e), \quad (4.4)$$

where $p_e \in [0, 1]$ is a teacher probability that typically decreases as training progresses.

This formulation interpolates between:

- **Teacher forcing** ($p_e = 1$): $\tilde{y}_{t-1} = y_{t-1}$,
- **Teacher-free autoregression** ($p_e = 0$): $\tilde{y}_{t-1} = \hat{y}_{t-1}$.

Under scheduled sampling, the effective training objective becomes an expectation over the stochastic injection process:

$$\min_{\theta} \mathbb{E}_{b \sim \text{Bernoulli}(p_e)} \left[\frac{1}{H} \sum_{t=1}^H \ell(y_t, \hat{y}_t(\theta; b)) \right] + \lambda \mathcal{L}_{\text{smooth}}. \quad (4.5)$$

Scheduled sampling limits error accumulation early in training by intermittently conditioning the model on ground-truth outputs, while gradually recovering a fully autoregressive regime as $p_e \rightarrow 0$. Although this procedure does not optimize the exact inference-time objective and may introduce bias, it often improves empirical convergence behaviour for long-horizon prediction tasks.

4.4.3 Teacher probability schedules. Let e denote the training epoch index. Common choices for the teacher probability schedule p_e include:

Linear decay

$$p_e = \max\left(0, 1 - \frac{e}{E_{\text{decay}}}\right), \quad (4.6)$$

where E_{decay} is the number of epochs over which teacher forcing is phased out.

Exponential decay

$$p_e = p_0 \alpha^e, \quad \alpha \in (0, 1), \quad (4.7)$$

which transitions more rapidly to the teacher-free regime.

In all cases, validation and test evaluation are performed with $p_e = 0$, corresponding to fully autoregressive inference.

4.4.4 Implementation in the training loop. In the present codebase, scheduled sampling is implemented inside the autoregressive regression routine, which rolls forward predictions over the window and injects the previous output into a designated input channel.

At each time step t , the injected value is computed per batch element as

$$\tilde{y}_{t-1}^{(b)} = \begin{cases} y_{t-1}^{(b)}, & \text{with probability } p_e, \\ \hat{y}_{t-1}^{(b)}, & \text{with probability } 1 - p_e, \end{cases}$$

where b indexes the batch dimension. This requires access to both the previous prediction $\hat{y}_{t-1}$ and the ground-truth output y_{t-1} during training.

The resulting predictions $\hat{y}_{1:H}$ are then stacked and compared to the ground-truth sequence to compute the loss, followed by back-propagation through the entire autoregressive unrolling.

4.4.5 Deterministic alternative. As an alternative to stochastic scheduled sampling, a deterministic convex combination can be used:

$$\tilde{y}_{t-1} = p_e y_{t-1} + (1 - p_e) \hat{y}_{t-1}. \quad (4.8)$$

This approach avoids stochasticity while retaining a gradual transition from teacher forcing to teacher-free autoregression. Although it departs from the original scheduled sampling formulation, it often yields smoother optimisation dynamics in practice.

4.4.6 Status. In the present implementation, training is performed in a fully teacher-free autoregressive regime. Scheduled sampling is identified as a natural extension to improve convergence stability, particularly in early training stages, without altering the inference-time behaviour of the model.

Related work. Scheduled sampling was originally introduced by Bengio et al. [1] as a practical strategy to mitigate the exposure bias arising from teacher-forced training of autoregressive sequence models. The method interpolates between conditioning on ground-truth outputs and model predictions during training, with the aim of progressively aligning the training distribution with the inference-time autoregressive regime.

The broader issue of training–inference mismatch in sequence models has been extensively discussed in the literature, notably in the context of sequence-level training objectives [2], where it is shown that local, teacher-forced losses may lead to suboptimal long-horizon behaviour.

Despite its empirical success, scheduled sampling has been shown to introduce bias and to be inconsistent with maximum likelihood training objectives [3], as the stochastic conditioning

mechanism alters the effective data-generating distribution seen during optimization. Several alternative approaches have been proposed to address this limitation, including adversarially enforcing consistency between training and inference dynamics, such as Professor Forcing [4].

In the present work, scheduled sampling is considered as a potential extension to improve early training stability in fully teacher-free autoregressive learning, while inference and evaluation are consistently performed in a self-feeding regime.

5 Hyperparameter Sensitivity and Trade-off Analysis

5.1 Purpose and Scope

The objective of this chapter is to empirically characterize the influence of architectural and training-related hyperparameters on both model performance and computational cost. Rather than identifying a single optimal configuration, the analysis aims to:

- quantify the sensitivity of validation loss and training time to individual hyperparameters,
- identify dominant sources of computational cost,
- expose trade-offs between performance and efficiency.

All results are based on a controlled hyperparameter sweep in which training conditions were kept fixed and only the selected hyperparameters were varied.

5.2 Experimental Setup

Each experiment corresponds to a single training run with a unique hyperparameter configuration. For every run, the following quantities were recorded:

- best validation loss achieved during training,
- total training time measured in seconds.

The explored hyperparameter space is summarized as follows:

- number of layers $n_{\text{layer}} \in \{4, 8\}$,
 - number of attention heads $n_{\text{head}} \in \{2, 4, 8\}$,
 - embedding dimension $n_{\text{embd}} \in \{16, 32\}$,
 - batch size $\in \{64, 128\}$.
-

5.3 Global Relationships Between Hyperparameters, Loss, and Time

Figure 5.1 reports global scatter plots relating validation loss and training time to the explored hyperparameters. These plots provide an overview of the empirical landscape but do not control for interactions between hyperparameters.

Two trends are immediately apparent. First, training time increases markedly with model size, particularly with respect to the number of layers and attention heads. Second, validation loss exhibits a weaker and less monotonic dependence on individual hyperparameters, suggesting the presence of interaction effects.

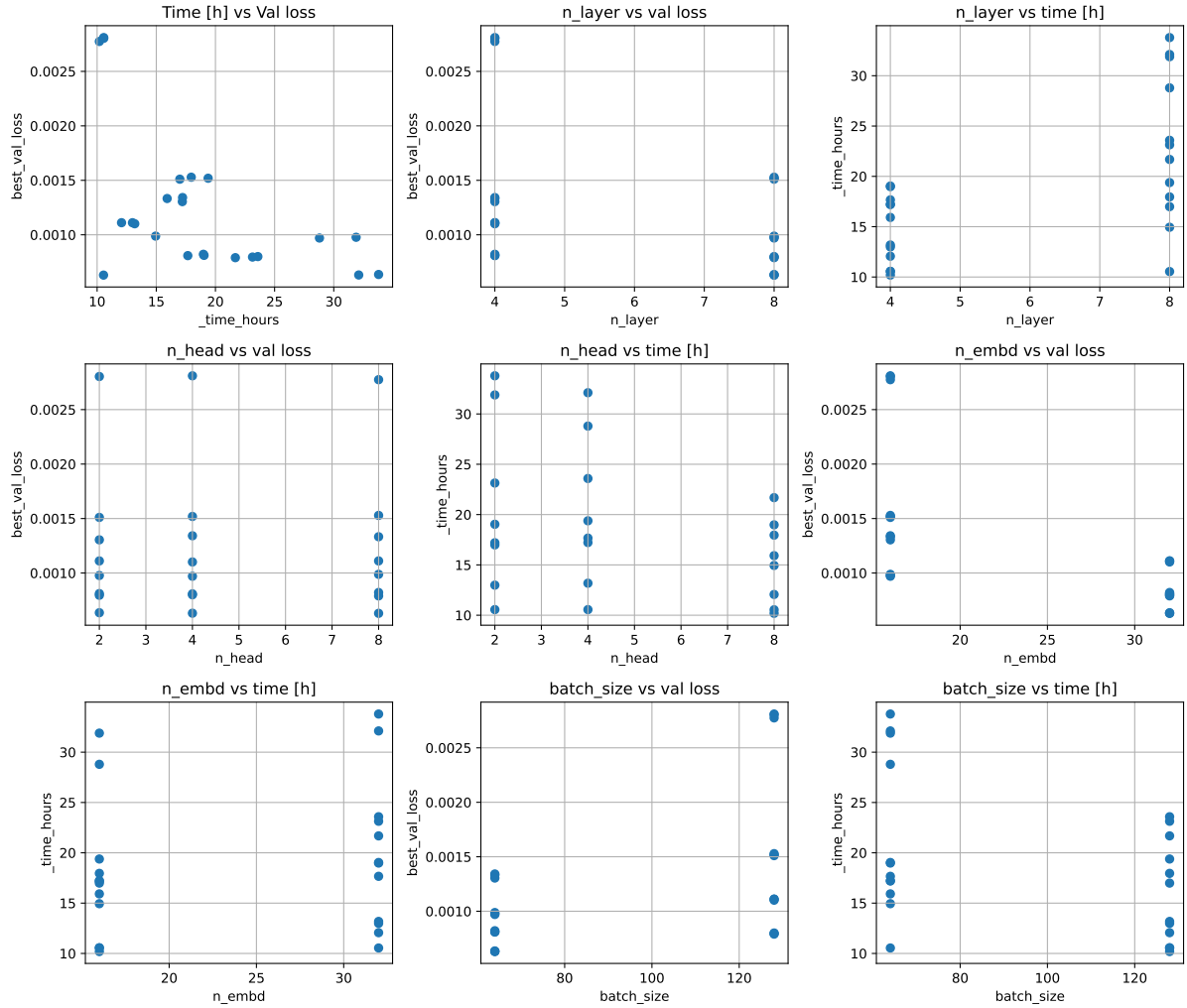


Figure 5.1: Global scatter plots illustrating the relationship between hyperparameters, validation loss, and training time across all runs.

5.4 One-at-a-Time Sensitivity Analysis

To isolate the effect of individual hyperparameters, a one-at-a-time sensitivity analysis was performed. For each hyperparameter, only pairs of runs differing in that parameter while sharing identical values for all others were compared.

5.4.1 Quantitative Summary. Table 5.1 reports aggregated statistics derived from the one-at-a-time comparisons.

Increasing the number of layers and attention heads consistently increases training time, whereas their effect on validation loss is comparatively weak. In contrast, the embedding dimension exhibits a stronger association with loss reduction, albeit at increased computational cost.

5.4.2 Slice-Based Visualization. Figure 5.2 visualizes one-at-a-time effects by plotting validation loss and training time as functions of each hyperparameter while keeping the remaining ones fixed.

The plots confirm that:

Table 5.1: Summary of one-at-a-time hyperparameter effects. Mean changes are computed over all valid comparisons.

Hyperparameter	Mean Δ loss	Mean Δ time [h]	Loss improvement rate
n_{layer}	< 0	> 0	moderate
n_{head}	≈ 0	> 0	low
n_{embd}	< 0	moderate	high
Batch size	≈ 0	mixed	low

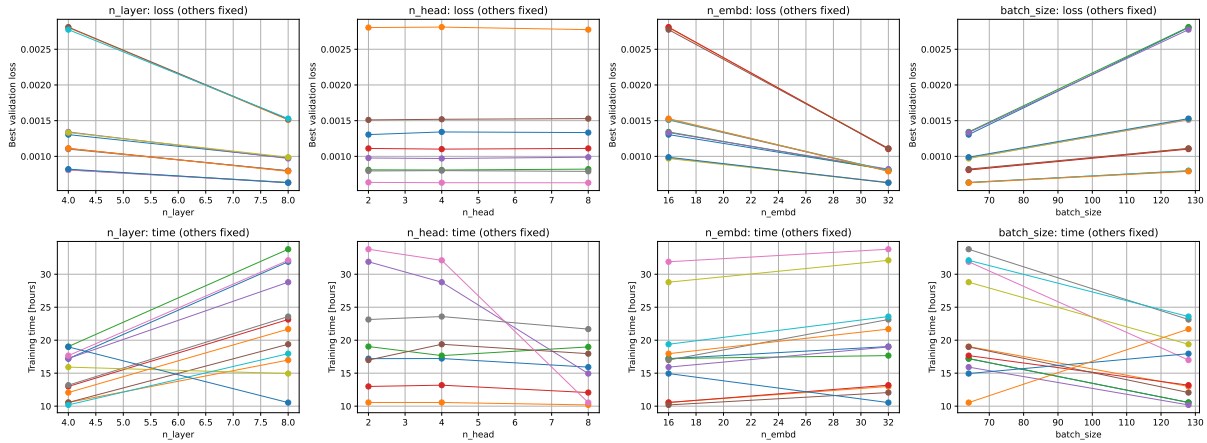


Figure 5.2: One-at-a-time sensitivity analysis. Each column corresponds to a hyperparameter; the top row reports validation loss and the bottom row training time.

- training time grows approximately monotonically with model depth and width,
- validation loss improvements saturate beyond moderate model sizes,
- batch size primarily affects training time rather than validation performance.

5.5 Trade-off Analysis and Pareto Optimality

Optimizing validation loss and training time simultaneously constitutes a multi-objective problem. To characterize the inherent trade-off, a Pareto front analysis was performed.

5.5.1 Pareto Front. Figure 5.3 shows the Pareto front in the validation loss versus training time plane.

Configurations on the Pareto front represent efficient trade-offs, as no simultaneous improvement in both objectives is possible. Notably, several intermediate-sized models achieve near-optimal validation loss with substantially reduced training time compared to the most expensive configurations.

Representative Pareto-optimal configurations are reported in Table 5.2.

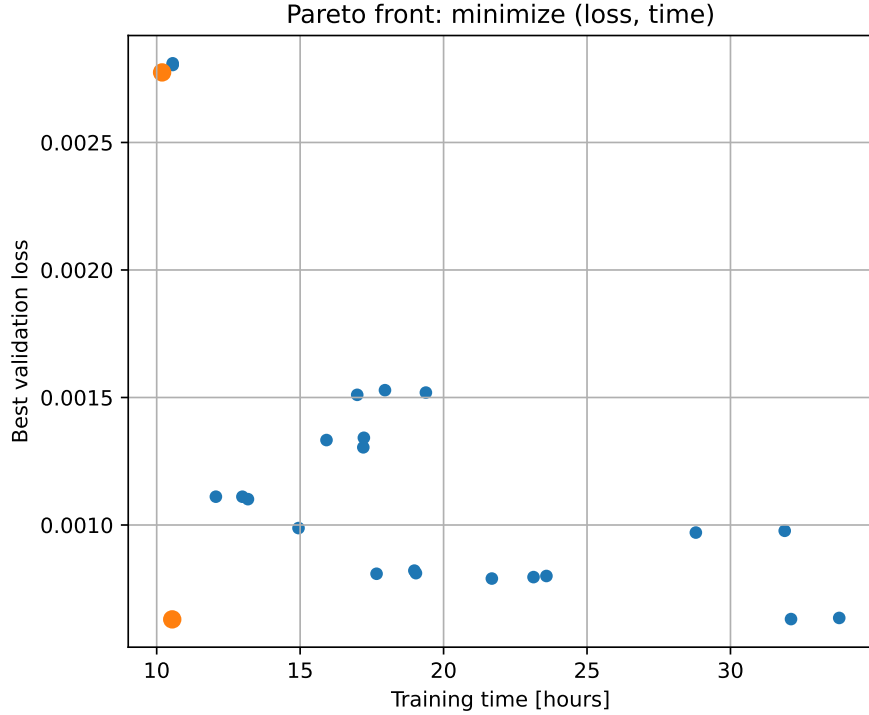


Figure 5.3: Pareto front of validation loss versus training time. Points on the front represent configurations that are not dominated in both objectives.

Table 5.2: Representative configurations on the Pareto front.

n_{layer}	n_{head}	n_{embd}	Batch	Val. loss	Time [h]
8	8	32	64	6.3×10^{-4}	10.5
4	8	32	128	1.1×10^{-3}	12.1
4	4	16	128	2.8×10^{-3}	10.6

5.5.2 Weighted Trade-off Selection. For practical model selection, a scalar trade-off score was defined as a weighted combination of normalized validation loss and training time. The weights reflect a preference toward validation performance while penalizing excessive computational cost.

Table 5.3 reports the top-ranked configurations according to this criterion.

Table 5.3: Top configurations according to the weighted trade-off score ($w_{\text{loss}} = 0.75$, $w_{\text{time}} = 0.25$).

n_{layer}	n_{head}	n_{embd}	Batch	Score	Time [h]
8	8	32	64	0.0037	10.5
4	4	32	64	0.141	17.7
4	2	32	64	0.156	19.0

It is emphasized that the trade-off score is preference-dependent and does not replace the Pareto analysis, but rather complements it by enabling a concrete selection under explicit assumptions.

5.6 Discussion

The hyperparameter sweep enables a direct comparison between the initially adopted default configuration and the configurations identified as more favorable through empirical analysis. The default setup, used as a baseline prior to the sweep, was

$$(n_{\text{layer}}, n_{\text{head}}, n_{\text{embd}}, \text{Batch}) = (8, 4, 16, 128).$$

Across the explored hyperparameter space, the configuration

$$(n_{\text{layer}}, n_{\text{head}}, n_{\text{embd}}, \text{Batch}) = (8, 8, 32, 64)$$

emerges as the most favorable trade-off between validation performance and computational cost. This configuration achieves the lowest validation loss observed in the sweep while maintaining a training time comparable to, or lower than, several less accurate alternatives.

The comparison between the default and selected configurations highlights two key effects. First, increasing the embedding dimension from 16 to 32 yields a substantial reduction in validation loss, indicating that representational capacity constitutes a primary performance bottleneck in the default setup. Second, increasing the number of attention heads from 4 to 8 improves performance without incurring a proportional increase in training time, suggesting that parallel attention mechanisms are exploited efficiently by the model and training implementation.

Conversely, the reduction of the batch size from 128 to 64 contributes to improved generalization at the expense of a moderate increase in gradient update frequency, while not significantly degrading overall training time. This observation is consistent with the weak dependence of validation loss on batch size observed throughout the sweep.

Overall, the results indicate that the default configuration underutilizes model capacity in terms of embedding dimension and attention width. The selected configuration represents a better-balanced operating point, achieving improved validation performance without excessive computational overhead. This configuration is therefore adopted as the reference setting for subsequent experiments, while the default configuration is retained solely as a historical baseline for comparison purposes.

5.7 Ablation on Regression Mode, Activation Function, and Smoothness

Table 5.4 reports the results obtained by varying the regression mode, activation function, and the inclusion of smoothness regularization. For each configuration, the best validation loss and total training time are reported.

Table 5.4: Ablation study on regression mode, activation function, and smoothness regularization.

Smoothness	Regression mode	Activation	Best val. loss	Training time
False	time_full	SiLU	6.89×10^{-4}	10h 39m
False	time_full	GELU	6.48×10^{-4}	10h 37m
False	time_last	SiLU	6.75×10^{-4}	10h 41m
False	time_last	GELU	7.03×10^{-4}	10h 44m
True	time_full	SiLU	7.75×10^{-4}	10h 37m
True	time_full	GELU	7.41×10^{-4}	10h 33m
True	time_last	SiLU	7.68×10^{-4}	10h 40m
True	time_last	GELU	7.64×10^{-4}	10h 41m

Discussion. The observed differences in validation loss across regression modes (`time_full` vs. `time_last`) are on the order of 10^{-5} – 10^{-4} and do not exhibit a consistent trend. Similarly, the choice of activation function (GELU vs. SiLU) leads to marginal and non-systematic variations in performance. Training time remains effectively unchanged across all configurations.

Enabling smoothness regularisation slightly increases the validation loss in all cases, reflecting the expected trade-off between pointwise accuracy and trajectory regularity. Importantly, no significant interaction effects between smoothness, regression mode, and activation function are observed.

Overall, these results indicate that neither the regression mode nor the activation function has a material impact on performance in the considered regime. Consequently, the regression mode is fixed to its default value `time_last` for conceptual simplicity and consistency with the autoregressive formulation, while GELU is retained as the default activation function.

Bibliography

- [1] S. Bengio, Y. Bengio, N. Giguère, and H. Larochelle, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Advances in Neural Information Processing Systems*, 2015.
- [2] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba, “Sequence level training with recurrent neural networks,” in *International Conference on Learning Representations*, 2016.
- [3] F. Huszár, “How (not) to train your generative model: Scheduled sampling, likelihood, adversary?” *arXiv preprint arXiv:1511.05101*, 2015.
- [4] A. Lamb, A. Goyal, Y. Zhang, S. Zhang, A. Courville, and Y. Bengio, “Professor forcing: A new algorithm for training recurrent networks,” in *Advances in Neural Information Processing Systems*, 2016.